Master prompt: build an exam knowledge system

A complete, subject-agnostic brief for an AI assistant. Give this document to a capable model with study material and it can build and run the same system for any exam subject — current affairs, quantitative aptitude, reasoning, English, static GK — without further instruction.

Written from a working build for SSC CGL and UPSC current affairs: 651 filed facts, 262 capsule points and 345 quiz questions across seven newspapers, served as a public website and JSON API.

1. What you are building

A single pipeline that turns raw study material into three linked products, so the learner can skip the raw material entirely and still be exam-ready:

Product	What it is	Why it exists
The web	Every exam-worthy item filed on a tree of subjects and topics, each with a permanent id, an exam tag, a priority and its source	Long-term memory. Nothing is ever lost or duplicated; a topic's whole history is one click away
The capsule	One date's or one chapter's material compressed to one-line points, grouped by subject, with the small exam details hidden behind a tap	The 5-10 minute read that replaces the raw material
The quiz	Questions built from the capsule, answers hidden until chosen, with an explanation and 1-2 related facts under each answer	Turns reading into recall; proves the capsule was enough

The rule that holds it together: the capsule must be sufficient to answer the quiz, and the quiz must cover every capsule point. Both directions are enforced by a script, not by good intentions.

2. The core concept

Material → **atomic items** on a topic web → **priority** from real exam evidence → **capsule** (compressed, with detail behind a tap) → **quiz** written from the capsule → **automatic checks** → publish → short report to the learner.

Four principles make it work for any subject:

Principle	What it means in practice
Atomise	One item = one testable idea, in 1-3 sentences, with the testable terms in **bold**. Bold terms become quiz answers and search keys.
Place, don't pile	Every item hangs on a topic node in a shallow tree (subject → topic → sub-topic). A continuing story or a recurring question type grows its own node, so the learner sees a timeline, not a feed.
Rank by evidence	Priority comes from previous years' papers, not from how interesting the item feels. Everything the learner sees can be filtered by it.
Compress, then test	The capsule is the deliverable; the web is the archive; the quiz is the proof. If an answer is not in the capsule, the capsule is wrong — not the learner.

3. Before you write anything: learn the exam

Do this first, once per subject. It decides every priority tag afterwards.

1. Collect **3-5 years of real previous-year papers** (official papers beat memory-based reconstructions; 8-10 shifts per year beats one shift from ten years).
2. Extract only the section you are building for. Classify every question: topic, what exactly is asked (a name, a number, a place, a rule, a method), and how old the tested material was.
3. Write the findings down as a **pick list** and a **skip list** with a priority scale. Coaching blogs are a starting point, not evidence — check them against the real papers and prefer the papers when they disagree.
4. Re-read this list whenever the learner corrects a pick. Every correction becomes a rule.

Worked example (SSC CGL current affairs). The 2021 official papers showed 5-9 current-affairs questions per 25-question section, events 3-9 months old, and — the surprise — that small items (a PSU chief, a brand ambassador, a mascot, a 'first State to…', a book's author) are asked as often as the big stories. That single finding changed what counts as high priority.

4. Data model

Plain JSON files in a git repository are the source of truth. Everything else is generated. Keep the generated output out of the editing path so a bad edit can never corrupt the archive.

File	Holds	Key fields
nodes.json	The topic tree: subjects (the 10-14 roots) and topics under them	id, label, summary, parent, order, created, links (related topics in other subjects); roots add short, hue
sources.json	Every piece of material used, so any item can be traced	id, date or chapter, publication/book, edition, pages, link, added
items/<group>.json	The items themselves, grouped by month (or chapter)	id (permanent, e.g. f20261104-017), node, tags (which exam), priority, text, page, extra (true if you added it)
capsules/<key>.json	One capsule: sections by subject, each a list of points	junction, points[{text, facts, more}] — more holds the ids revealed on tap
quizzes/<key>.json	One quiz	id, fact (the item tested), type, q, options, answer, explanation, also

- **Ids are permanent.** Never renumber. A corrected item keeps its id, so quiz progress and links never break.
- **Tags and priority live on the item**, never on the question or the capsule point, so the three products can never disagree with each other.
- **Mark your own additions.** Anything not present in the source material is flagged (extra: true) and shown as a note, never as something the source said.

5. The four tools

Write these before filing anything. They are what keeps the daily cost flat as the archive grows.

Tool	Does
add_batch	Takes **one spec file per batch of material** (the source, new topics, items, corrections to older items, the capsule, the quiz) and does everything: numbers ids, fills topic order and creation date, adds links both ways, shuffles answer options, runs the build, and **restores every file if anything fails**. Refuses a source that was already added.
build	Validates the whole archive, then regenerates the API and the spreadsheet exports. Prints OK or a list of errors and writes nothing. Validation must include: ids unique and well-formed, every item on an existing topic, every required tag present, priority present where required, options distinct, answer in range, cross-references real.
find	One-line search over items, topics and questions. Used before filing, to catch duplicates and to spot a continuing story without reading whole files.
check_capsule	The quality gate: every quiz answer must be findable in that capsule, and every capsule point must have at least one question. Prints the gaps; exits non-zero.

Why a single spec file matters. The first version of this system used a fresh hand-written script per batch. By the third batch that was the slowest part of the work and the only place bugs appeared. One declarative spec plus one idempotent command removed both problems.

6. The pages

Page	Must do
Capsule (the home page for daily use)	Pick a date or chapter; points grouped by subject; tap the circle to mark read; tap a point to reveal the exam notes; reading-time estimate; one button to the matching quiz; link to the source.
Quiz	Same picker; filters by exam and priority; options hidden until one is chosen; then the verdict, a one-line explanation, 'also remember' facts, the source and a link to the topic; a running score.
Web	The whole tree, searchable and filterable, with each topic's items newest-first — the archive view.
Sources	Every piece of material used, with a link to the original and how many items came from it.
API	The same data as plain JSON so a future app (flashcards, a bot, spaced repetition) needs no scraping.

- Load only what the current view needs (one small file per date) so the pages stay fast after a year of material.
- Store the learner's ticks and scores locally in the browser; never make reading depend on a login.
- Light and dark themes, readable on a phone, no horizontal scrolling.

7. Writing rules that decide quality

Items

- One idea per item. Continuing stories get an update phrased as an update, never a repeat.
- Bold every testable term: names, numbers, dates, places, section numbers, formulas.
- Be faithful to the source. Attribute claims (“the minister said…”). Never present your own knowledge as the source's.
- Add the background the source leaves out — but only through one test: **would a question on this detail plausibly appear in the exam?** Cap it at about two per story. Unchecked background is the fastest way to ruin the capsule.

Capsule points

- One line per story or concept, under ~240 characters, grouped by subject; fold the background into the line or park it behind the tap.
- Cover every high-priority item; drop the low-priority ones entirely. Target a 5-10 minute read.
- Write the detail the learner must *recall*, not the detail that makes the sentence read nicely.

Questions

- Walk the capsule point by point so nothing is left untested. Ask exactly what the exam asks.
- Distractors must be the same kind of thing as the answer (four holders of the same office, four close years, four States of one region). Never joke options — they teach nothing and inflate scores.
- Spread correct answers evenly across the positions; shuffle options programmatically, not by feel.
- Add 1-2 related facts under the answer, chosen by hand from the same story. Loosely related extras are worse than none.
- Mix formats the exam uses: direct, 'which statements are correct', assertion-reason.

8. Adapting the concept to other subjects

Only the meaning of an *item* changes. The pipeline, the tools and the checks stay identical.

Subject	An item is…	Topic tree	Priority from	Capsule looks like	Quiz
Current affairs	A fact with its source and date	Subjects → running stories	Frequency of that fact type in past papers	One line per story	Recall MCQs
Quantitative aptitude	A method, formula or trap, with one worked example and the condition it applies under	Arithmetic / algebra / geometry / DI → question types	How many marks that type carried per paper	The formula plus the one-line trigger: 'when the ratio is given and time differs, use…'	Solve-it items generated from the same template with new numbers; the worked example is the explanation
Reasoning	A pattern with its recognition cue and the fastest route to the answer	Verbal / non-verbal → puzzle families (seating, syllogism, series…)	Count of each family per shift	Cue → method → common trap, one line each	One fresh puzzle per family; time limit per item

Subject	An item is…	Topic tree	Priority from	Capsule looks like	Quiz
English	A rule, a confusable pair, an idiom or a high-frequency word	Grammar / vocabulary / comprehension → rule families	Error-spotting and vocabulary repeats	Rule plus a minimal example and the wrong version	Spot-the-error and fill-in from the same rule
Static GK, history, polity	A fact tied to a syllabus heading, not a date	Syllabus headings → sub-headings	Syllabus weight and past-paper counts	Grouped tables (rulers, articles, dams, dances)	Matching and direct recall
Science	A concept with its definition, unit and one application	Physics / chemistry / biology → chapters	Past-paper counts per chapter	Definition, formula, one application	Direct recall plus one applied item

Two adaptations worth making explicit. For skill subjects (maths, reasoning) add a *variant generator*: each item carries a template so the quiz can produce unlimited fresh questions with the same method, and record which methods the learner keeps failing. For subjects with a fixed syllabus, index the tree by the official syllabus headings so coverage gaps are visible at a glance.

9. Build order

1. **Study the papers** (section 3) and write the pick, skip and priority rules into a document you will follow every day.
2. **Create the repository** and the empty data files; decide the subject roots and their colours.
3. **Write build first**, with full validation, then find, then add_batch, then check_capsule. Test add_batch on a copy of the repository, including a deliberately broken spec, before trusting it.
4. **File one batch of material end to end** — items, capsule, quiz — and fix what the checks complain about.
5. **Build the pages**, capsule first; publish; verify the deployment actually serves the new data before telling the learner.
6. **Write the daily procedure down** (as a reusable instruction file) so a fresh session repeats it exactly.

10. Acceptance checklist

A batch is finished only when all of these are true:

- The build prints OK and the checker reports **0 gaps and 0 untested points**.
- Every item carries a priority (where the exam tag requires one) and traces to a source with a page or location.
- The capsule reads in the target time and covers every high-priority item.
- Correct answers are spread across the option positions; no duplicate or joke distractors.
- The deployment is live and serving the new data — verified, not assumed.
- The learner gets a short report: what was added, what was skipped and why, what changed, what is still missing.

11. Operating rules

- **Version everything.** A git repository plus a static host costs nothing and keeps the full history; the learner should never depend on a single chat or a single device.
- **Never commit copyrighted source material.** Link to the learner's own copy instead, and tell them to keep the sharing restricted.
- **Correct in place.** When later material shows an earlier item was wrong, fix that item (keeping its id) and say so in the report. Silent corrections destroy trust in the archive.
- **Report honestly.** Say what was skipped and what could not be verified. A confident summary over a thin pass is the one failure mode the learner cannot detect.

12. The prompt to paste

Give the model this, together with the material:

```
You are building and running an exam knowledge system for <exam>, subject <subject>.

First, study the previous-year papers I give you and write down, as rules, what that exam
actually asks: the question types, how they are phrased, and which details are tested. Rank
everything you will later file as high, medium or low priority on that evidence alone.

Then, for every batch of material I send, run this pipeline without being asked:
1. Read all of it, including the parts that look unpromising.
2. File each exam-worthy item on a topic tree, with a permanent id, an exam tag, a priority,
the testable terms in bold, and the source and page.
3. Add the exam background the material leaves out, but only details a question could
```

```
plausibly be built on, and mark them as your own additions.
4. Write a capsule: that batch compressed to one-line points grouped by subject, a
five-to-ten minute read, with the small details hidden behind a tap.
5. Write the quiz from the capsule, point by point, so every point is tested and every answer
is findable in the capsule. Hide the answers until chosen; give each one a one-line
explanation and one or two closely related facts.
6. Attach the source link, run your own checker until it reports no gaps in either direction,
publish, and verify the published copy.
7. Report briefly: what was added, what was skipped and why, what you corrected, what is still
missing.

Build the tools so that each batch is one spec file and one command, with automatic rollback.
Keep everything in a git repository with a static site and a JSON API. Never commit the source
material itself. Never present your own knowledge as the source's. When I correct a judgement
of yours, write it into the rules so it holds next time.
```

Everything in this document was derived from a working system, not a plan. The parts that proved most valuable, in order: priority judged from real papers; the capsule as the primary product; and the two-way check between the capsule and the quiz — which caught four unanswerable questions on the first day it existed.